

Instructor Handout 6: Hardwired Control Unit Design

Auqib Hamid Lone

Department of Computer Science & Engineering, GCET Ganderbal

August 8, 2026

Instructor Copy — Not for Student Distribution

6.1 Before You Teach This Lecture

- Mano, *Computer System Architecture*, 3rd ed., Sec. 5-4 “Timing and Control,” Sec. 5-9 “Control of Registers and Common Bus,” Sec. 5-10 “Control of the Common Bus and Memory,” pp. 135–167 (indexed, page-cited; corrected 2026-08-06 from an earlier draft’s mis-attribution to Ch. 7, which is actually Week 7’s microprogrammed-control material). Read Fig. 5-6, p.137 once before class — the two-decoder-plus-control-matrix block diagram below is styled directly on it, and knowing where the book’s own labels sit will make it easier to field a “why does the book draw it differently” question.
- Hamacher, *Computer Organization*, Ch. 7 — *not yet page-indexed*. Cite at chapter level with students; it’s the source for the control-unit input/output contract used below.
- Reread Week 5’s single-bus trace ($R1 \leftarrow R2+R3$, T_1, T_2, T_3 via Y/Z) before this lecture — this entire lecture is that trace’s missing piece, made mechanical, and the opening minutes lean on students remembering it cold.

6.2 Control Unit Functions

Open by returning to the exact three-line trace from last week and asking the class directly: Y loaded from the bus at T_1 and Z loaded from the ALU at T_2 — never the other way round. Who decided that ordering? Not the ALU (it only computes what it’s told), not the bus (it only carries what’s driven onto it), not the register file (it only loads/drives when signaled). Every one of those components is *passive*. Something else has to watch the instruction and the clock and tell every other component what to do, every cycle.

Definition (Control Unit). *The control unit is the hardware that, every clock cycle, examines the current instruction and the current control step, and asserts the **control signals** that make every other component (registers, ALU, memory, buses) do the right thing for that instruction at that step.*

Be precise about the boundary: the control unit does not compute results (the ALU’s job) and does not store data (registers’/memory’s job) — it only decides *when* and *which* component acts.

How to present it: draw the central box and label it first, empty. Add the four input boxes one at a time on the left, asking after each “what would the control unit need to know this for?”

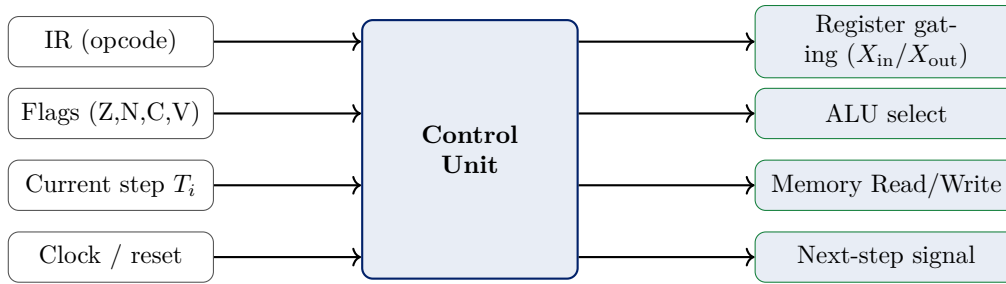


Figure 6.1: The control unit as a black box: inputs (left) row-aligned with the outputs (right) they most directly drive — cf. Hamacher, Ch. 7.

before revealing the matching output on the right at the *same height* — the row-alignment is the whole point of the figure, so build it row-by-row, not column-by-column.

Teaching note: Students often think the control unit “computes” something. Head this off directly: it routes decisions, it does not do arithmetic. A quick contrast question works well here — “if the control unit breaks, does the ALU stop being able to add?” (No — it just never gets told to.)

Introduce the $X_{\text{in}}/X_{\text{out}}$ notation next, applied immediately to last week’s trace: at T_1 , $R2_{\text{out}}$ and Y_{in} fire together; at T_2 , $R3_{\text{out}}$ fires, ALU is set to add, and Z_{in} fires; at T_3 , Z_{out} and $R1_{\text{in}}$ fire together. State plainly: these three lines are exactly what the rest of the lecture builds hardware to generate automatically.

Then contrast the two design philosophies in one breath, before diving into either: hardwired control wires the (opcode, step) $\rightarrow$ signal mapping directly into gates (fast, fixed once built); micro-programmed control stores it in memory and steps through it (slower per step, easy to change). This lecture builds hardwired in full; Week 7 rebuilds the identical control unit as microprogrammed, for direct comparison.

6.3 The Sequence Counter and Timing Generation

Pose the gap explicitly: last week, T_1, T_2, T_3 were written *by hand*, on paper. Nothing in the hardware yet decides “it is now T_2 .” What physical component would track that?

Definition (Sequence Counter (SC)). A small counter register holding the number of the current control step. A **step decoder** converts SC’s binary value into one-hot signals $T_1, T_2, T_3, \dots$, so exactly one T_i line is active per cycle.

How to present it: draw SC alone first and ask “if this just counts 0, 1, 2, 3, $\dots$ in binary, how does that become a single active T_2 wire?” *before* drawing the decoder — most classes will guess “some kind of decoder” unprompted, which makes the reveal land as confirmation rather than new information. Draw the decoder next, then the $T_1/T_2/T_3$ output lines one at a time, tracing which decoder output line corresponds to which SC value.

Teaching note: Flag the T_0 -vs- T_1 indexing mismatch with Mano explicitly and early — if a student opens the book and sees T_0 where the board says T_1 , that is not an error, it’s a course convention choice (this course numbers from 1). Naming the discrepancy once, out loud, prevents three separate confused questions later.

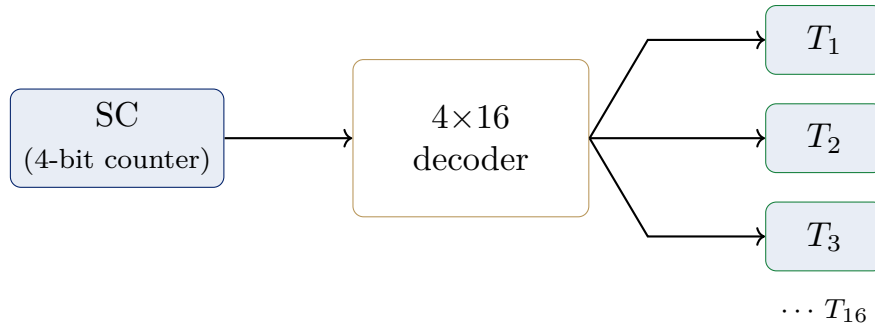


Figure 6.2: The sequence counter feeding a step decoder — cf. Mano, Fig. 5-6, p.137. This course numbers steps from T_1 ; Mano’s own text numbers from T_0 .

Explain SC ’s advance rule: by default $SC \leftarrow [SC] + 1$ every clock pulse; at the instruction’s *last* step, the control unit asserts SC_{clear} , resetting $SC \leftarrow 0$. Ask the class live, before answering: “what would happen if we forgot to wire up SC_{clear} ?” Let a few guesses land, then confirm: SC would keep counting $T_4, T_5, \dots$ forever, the decoder would produce one-hot lines for steps no instruction defines any signals for, the datapath would simply idle, and the CPU would freeze after the very first instruction.

Example 6.3.1 (Worked Trace Revisited: Where SC Points). T_1 : $SC = 0, Y \leftarrow [R2]$. T_2 : $SC = 1, Z \leftarrow [Y] + [R3]$. T_3 : $SC = 2, R1 \leftarrow [Z]$, then $SC_{clear} = 1$ fires, resetting $SC \leftarrow 0$.

Example 6.3.2 (A Second Instance: A 5-Step Instruction). Suppose a hypothetical *LOAD* instruction needs 5 steps (memory access is slower than a register-only *ALU* op). SC counts 0, 1, 2, 3, 4 across T_1 – T_5 , and SC_{clear} fires at T_5 , not T_3 — worth stating explicitly that SC_{clear} ’s firing point is instruction-specific, decided by each instruction’s own state table (Section 6.5), not fixed hardware.

6.4 Opcode Decoding and the Control Matrix

SC answers *when*. It says nothing about *which* instruction is running — Y_{in} should fire at T_1 for *ADD*, but a *LOAD* needs something else entirely at its own T_1 . Introduce D_i : a one-hot opcode-decoder output, $D_{ADD} = 1$ exactly when *IR* holds the *ADD* opcode. State the key structural fact plainly, as a claim the rest of the lecture will prove: every control signal depends on *both* an opcode line and a step line together, never either alone.

How to present it: build the two decoder chains independently and in parallel columns — $IR \rightarrow 3 \times 8$ decoder on top, $SC \rightarrow 4 \times 16$ decoder on bottom — before drawing the control matrix box at all. Only once both output bundles (D_i ’s and T_j ’s) are on the board do you draw the matrix and route both bundles into it, plus the I flip-flop fed from *IR* bit 15. This ordering matches how the two chains are conceptually independent until they converge.

Teaching note: The most common error here is students treating the opcode decoder and step decoder as one combined circuit rather than two entirely separate decode operations that happen to feed the same downstream box. Ask directly: “does the step decoder need to know the opcode to produce T_2 ?” (No.) That question, asked before revealing the matrix, tends to fix the misconception before it forms.

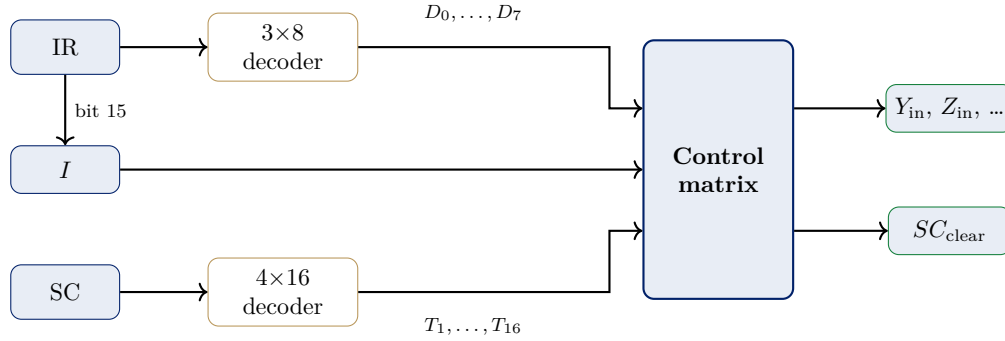


Figure 6.3: Two decoders feeding the control matrix — cf. Mano, Fig. 5-6, p.137. IR bits 0–11 also feed the control logic gates directly.

6.5 Deriving the Control Equations

This is the section where the lecture stops gesturing at "the control unit decides" and produces actual logic gates. Introduce the state table as the bridge.

Definition (State Table). Lists, for each control step, which control signals are active. A signal is asserted only when both the relevant opcode line $D_i = 1$ and the listed step $T_j = 1$ hold simultaneously.

Step	RTL	Control signals active
T_1	$Y \leftarrow [R2]$	$R2_{out}, Y_{in}$
T_2	$Z \leftarrow [Y] + [R3]$	$R3_{out}, ALU=add, Z_{in}$
T_3	$R1 \leftarrow [Z]$	$Z_{out}, R1_{in}, SC_{clear}$

Read the equations straight off this table on the board:

$$Y_{in} = D_{ADD} \cdot T_1, \quad Z_{in} = D_{ADD} \cdot T_2, \quad R1_{in} = D_{ADD} \cdot T_3, \quad SC_{clear} = D_{ADD} \cdot T_3.$$

Example 6.5.1 (Deriving Z_{in} for Two Instructions). Suppose *SUB* also uses Z at its own T_2 (identical 3-step shape to *ADD*, differing only in the ALU function at T_2). From *ADD*: $D_{ADD} \cdot T_2$. From *SUB*: $D_{SUB} \cdot T_2$. Any instruction whose state table lists Z_{in} at T_2 contributes a term, OR'd in:

$$Z_{in} = (D_{ADD} \cdot T_2) + (D_{SUB} \cdot T_2) = T_2 \cdot (D_{ADD} + D_{SUB}).$$

Teaching note: Derive this one live, step by step, rather than presenting the final OR equation directly — write *ADD*'s term, ask "what happens if *SUB* needs the same signal," then write *SUB*'s term, then ask how to combine them *before* writing the OR. The general pattern (every control signal is an OR, across instructions, of $D_i \cdot T_j$ terms) lands much harder as something the class derives than something they're told.

How to present it: draw the two AND gates first, each fed by its own (D_i, T_2) pair, and say explicitly "this AND gate is the Boolean equation's first product term, drawn as hardware" — then draw the OR gate and connect both AND outputs into it, narrating that the equation and the physical layout are the *same object*, viewed two ways. This is the moment that makes "hardwired" concrete rather than a vocabulary word.

Close by assembling the full capstone diagram (Figure 6.5) and running the "adding *MUL*" thought experiment live: a new opcode line D_{MUL} must be added to the decoder; every signal *MUL* touches (e.g. Z_{in}) must be re-derived; and a new AND gate, $D_{MUL} \cdot T_2$, must be physically wired into the existing OR gate. None of this is a software change.

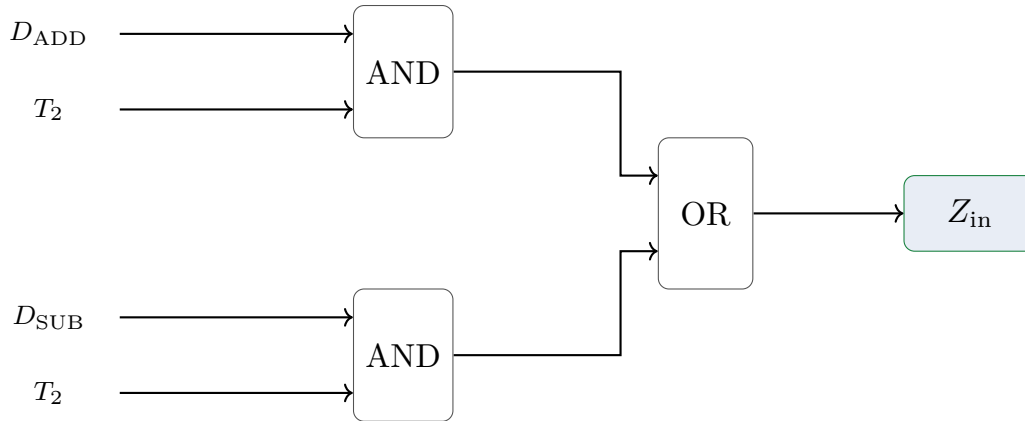


Figure 6.4: The control matrix as actual logic gates realizing $Z_{in} = (D_{ADD} \cdot T_2) + (D_{SUB} \cdot T_2)$.

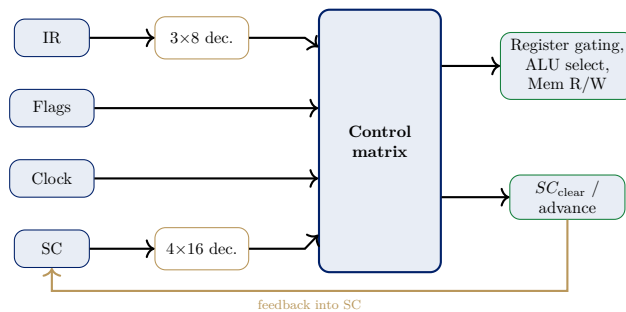


Figure 6.5: The complete hardwired control unit — cf. Mano, Fig. 5-6, p.137, extended with explicit Flags/Clock inputs and the SC feedback loop.

Teaching note: Draw the SC_{clear} feedback arrow *last*, and narrate it as closing a loop: “the control matrix’s own output decides whether the very counter that feeds the control matrix keeps counting or resets.” Students who see the feedback arrow before understanding the forward path tend to read the diagram as more complicated than it is.

6.6 Summary — Quick Reference

Control unit: reads (opcode, step, flags), asserts control signals every cycle. **SC + step decoder:** generates $T_1, T_2, \dots$ automatically; SC_{clear} resets at the last step. **Opcode decoder:** one-hot D_i lines. **Every control signal:** OR, across instructions, of $(D_i \cdot T_j)$ AND terms — state table makes this systematic. **Hardwired control pipeline:** state table $\rightarrow$ Boolean equations $\rightarrow$ AND/OR gates — fast, but new instructions mean new gates wired by hand, which is exactly what motivates microprogrammed control (Week 7). Control-step count is not free: single-bus needs 3 states, three-bus needs 1 — design cost simply moves between bus wiring and control logic.